

COMP90024 Assignment 2 Team 36 Report

Analysing Australian Fuel Prices and Cost-of-Living Sentiment

Xinyao Li (1560643) Hanyue Li (1650115) Junyao Zhang (1564528)

Hongkun Zhang (1638502) Yichen Sun (1682096)

GitLab Repository: https://gitlab.unimelb.edu.au/JUNYAOZ5/comp90024_team_36.git

YouTube Demonstration: <https://youtu.be/jXyvtCNpm1c?si=aDljLsISkSJX92Db>

Abstract

Fuel prices are a visible part of everyday financial pressure in Australia, because changes at the pump are noticed quickly by households. This project investigates whether those price changes are reflected in online discussion about fuel and cost-of-living issues. We built a cloud-based analytics pipeline that collects fuel price records and social media posts, stores them in Elasticsearch, and exposes aggregated results through a RESTful API for analysis in a Jupyter Notebook.

The system uses three data sources: FuelWatch WA for official daily fuel price records, Reddit for longer Australian community discussions, and Bluesky for shorter real-time posts. Automated ingestion is handled by Fission functions running on Kubernetes, with Elasticsearch used as the main storage and aggregation backend.

The results show that fuel prices are positively associated with discussion volume (correlation = 0.488), suggesting that higher prices tend to coincide with increased online discussion. The relationship between fuel prices and sentiment is much weaker (-0.082), including across a seven-day lag window. The two platforms also behave differently in the collected data: Reddit has higher average sentiment (0.278 compared with 0.029), while Bluesky contributes more posts overall (45,489 compared with 27,151).

1. Introduction

1.1 Project Background

Fuel prices are a direct and visible cost for many Australian households. Unlike broader indicators such as inflation or interest rates, petrol prices are seen regularly on roadside signs and affect weekly transport spending. Because many people depend on cars for commuting, work, and everyday activities, changes in fuel prices can quickly become part of cost-of-living pressure.

In recent years, supply disruptions, geopolitical instability, and inflation have kept living costs as a major public concern. Fuel prices are part of this issue because transport is difficult to avoid, especially for suburban and regional residents. As a result, price changes at the pump are not only an economic signal, but also a practical household concern.

Social media provides a useful way to observe public reaction to these changes. Platforms such as Reddit and Bluesky contain timestamped discussions where users comment on fuel prices, transport costs, and broader financial pressure. This project uses that opportunity to collect both fuel price data and online discussion data, and to analyse how they move in relation to each other.

1.2 Project Objectives and Data Sources

The main objective of the system is to automate the collection of fuel price data and social media posts, store them in a searchable backend, and support later analysis through a Notebook interface. The main analytical question is whether changes in fuel prices are associated with changes in online discussion volume or sentiment.

The project uses three data sources. FuelWatch WA provides official daily retail fuel price records by station. Reddit provides longer Australian community discussions, especially from subreddits such as r/australia and r/AusFinance. Bluesky provides shorter and more immediate posts, making it useful for capturing real-time reactions. Using both Reddit and Bluesky allows the analysis to compare two different styles of online discussion rather than relying on a single platform.

1.3 Report Structure

Section 2 covers system scenarios, analytical questions, and key results. Section 3 describes the overall architecture, including ingestion pipelines, storage design, and the API and visualisation layers. Section 4 covers implementation details and deployment. Sections 5 and 6 address scalability and technology trade-offs respectively. Sections 7 and 8 cover error handling, limitations, and testing. Section 9 concludes the report.

2. System Scenarios and Functionality

2.1 Scenario Motivation and Research Context

Recent geopolitical turmoil and volatility in global energy markets—factors increasingly tracked via global event databases (Hong et al., 2025)—have exacerbated fuel price volatility in Australia, further raising public concerns about transport costs and cost-of-living pressures. In recent years, Australians have become increasingly concerned about the issue of the cost of living, and fuel prices have become one of the most prominent and frequently discussed economic indicators affecting daily life. Thus, fuel price volatility is not an isolated economic event, but an indicator closely related to public financial stress and consumer sentiment.

At the same time, social media has become an important place for people to share opinions, make complaints, and react to economic conditions. Surveys and polls are still useful, but they often take time to collect and release. Social media data can offer a faster view of public behavior at a much larger scale. Reddit and Bluesky, for example, allow researchers to follow public discussion as it develops. These platforms can help show how discussion volume, sentiment, and response patterns change over time.

This project aims to explore how fuel price changes affect the discussion and public opinion sentiment about the cost of living on Australian social media platforms. Unlike research approaches that rely solely on social media sentiment analysis, this project combines official fuel price data with unstructured online discussions to establish a stronger link between real-world economic conditions and public online behavior. FuelWatch data serves as an official and quantifiable economic reference point, while Reddit and Bluesky provide complementary public reaction data from different online communities.

The use of multiple social media platforms helps improve the robustness and diversity of the analysis. Reddit tends to contain longer, more detailed discussion threads, while Bluesky tends to reflect quick, short reactions to breaking events. Comparing these platforms provides a more complete picture of how different online communities respond to fuel price volatility and broader cost of living issues.

Another important research motivation for this project is the instantaneous nature of fuel price changes and public reaction. Fuel prices are constantly updated, while online discussions and public opinion change rapidly over time. This requires an automated and scalable analytics pipeline capable of continuously collecting, processing, and analysing multi-source data streams. To support this continuously changing data, the project uses Kubernetes, Fission, Elasticsearch, and RESTful services to automate ingestion, storage, and analytics workflows.

2.2 Supported Analytical Questions and Functionalities

Our analytics platform supports multiple forms of time series, behavioural and sentiment-based analysis related to fuel price volatility and cost of living discussions in Australia. By integrating official FuelWatch data with continuously collected Reddit and Bluesky content, the system enables users to explore the evolution of economic conditions and online public reactions over time.

The platform first supports fuel price trend analysis through daily fuel price visualization and rolling average smoothing. This feature enables the observation of long-term price movements and short-term fluctuations in the Australian fuel market.

In addition, the system supports the analysis of discussion trends across social media platforms. By comparing rolling discussion volumes with official fuel price trends, the platform can investigate whether rising fuel prices are linked to increased public discussions surrounding cost-of-living issues.

The platform also supports sentiment analysis on Reddit and Bluesky datasets. Users can analyse aggregated sentiment scores and rolling averages to investigate the relationship between public sentiment and fuel price fluctuations. In addition, the system also provides correlation and lagged correlation analysis for investigating potential delayed social media responses.

Cross-platform comparative analysis is another key function of the system. The platform supports comparisons between Reddit and Bluesky in discussion behavior, posting activity, and sentiment dynamics. This helps reveal differences in public reactions across different social ecosystems.

These functions allow the frontend Notebook to compare fuel price trends, discussion volume, and sentiment on the same daily timeline, supporting exploratory analysis of how online discussions respond to changes in fuel prices and living costs.

2.3 System Functionality from User Perspective

The platform provides an interactive Jupyter Notebook interface where users can explore collected social media and fuel price data through a visual analytics dashboard. Users can interactively browse the analysis results, compare datasets across platforms, and observe time-series trends related to fuel prices and cost of living discussions.

The system supports filtering and exploration of the collected data based on multiple dimensions, including social media platforms, time periods, sentiment categories, and discussion topics. These filtering features enable users to focus on specific events, time periods, or online discussion patterns of interest.

The platform also provides interactive visualisations for fuel price trends, discussion volume dynamics, sentiment changes, and cross-platform comparisons, which are crucial for making abstract sentiment scores interpretable to users (Jain et al., 2023). Rolling averages, correlation plots, and comparative charts are presented directly within the Jupyter Notebook frontend to support intuitive exploration of the collected datasets.

Since the frontend is connected to RESTful backend services and Elasticsearch storage, the newly collected data can be continuously integrated into the analytical workflow. This enables near real-time updates and supports scalable cloud-based analytics on continuously growing social media datasets.

2.4 Results and Visualisations

2.4.1 Analytical Data Preparation

After retrieving aggregated data from the RESTful backend API, the frontend pipeline performs data cleaning, type conversion, and platform-specific feature extraction using Pandas. Temporal fields are converted into datetime format, and numeric fields are standardized for consistency. The system also extracts platform-level sentiment and discussion statistics from nested JSON structures for subsequent analytical processing.

2.4.2 Fuel Price Trend Analysis

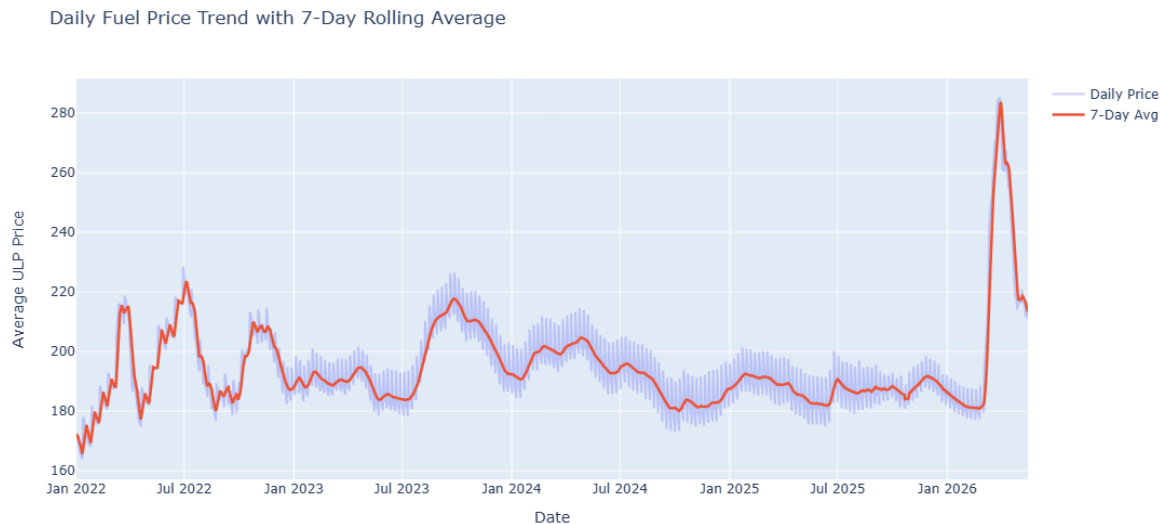


Figure 1. Daily Australian ULP fuel prices with 7-day rolling average smoothing

Figure 1 illustrates the daily Australian ULP fuel price trend together with a 7-day rolling average used to smooth short-term fluctuations. Fuel prices remained highly dynamic throughout the observed period, with several noticeable spikes occurring between 2022 and 2026.

The sharp fuel price surge observed in 2026 is consistent with broader global energy market instability and geopolitical tensions during that period. This sharp increase provides a useful period for comparing fuel price changes with shifts in online discussion activity and sentiment.

2.4.3 Social Media Discussion Volume Analysis

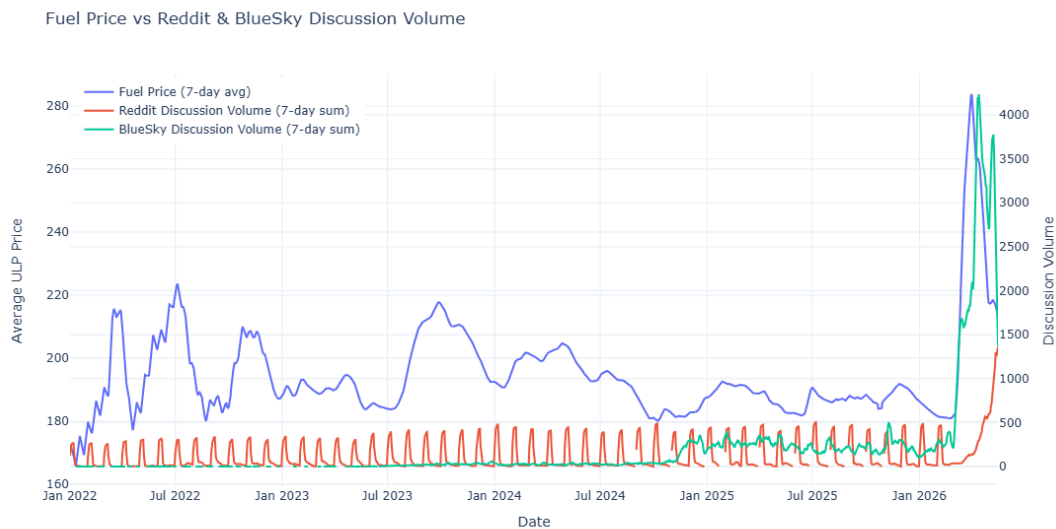


Figure 2. Comparison between fuel price trends and social media discussion volume across Reddit and Bluesky

Figure 2 compares official fuel price trends with discussion volumes collected from Reddit and Bluesky. Reddit discussion activity exhibits regular spike-like patterns during earlier periods because part of the historical Reddit dataset was derived from archived weekly aggregated records. After February 2026, the project transitioned to real-time Reddit data ingestion, resulting in smoother temporal behaviour.

Bluesky discussion volume remained relatively low before late 2024 due to the platform’s smaller user base during its early adoption stage. However, discussion activity increased substantially after late 2024 as the platform became more widely used. Overall, the figure shows that social media discussion activity increased during periods of elevated fuel prices, indicating growing online attention toward fuel-related economic issues. These temporal patterns provide the basis for the subsequent sentiment and cross-platform analyses presented later in the report.

2.4.4 Social Media Sentiment Analysis

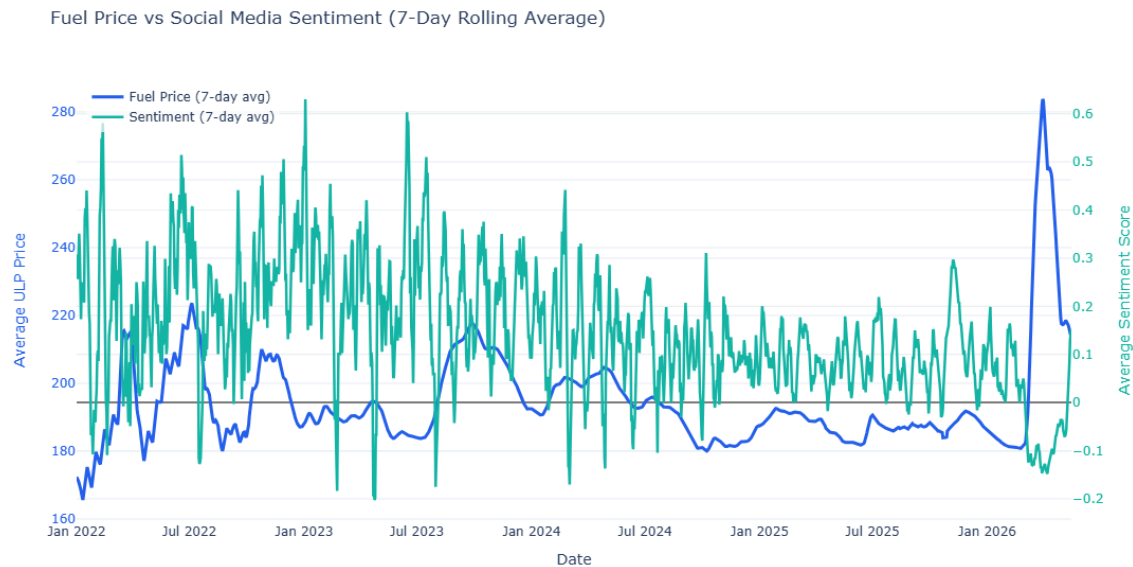


Figure 3. Fuel price trends and social media sentiment (7-day rolling averages)

Figure 3 compares the rolling average fuel price trend with aggregated social media sentiment over time. Overall sentiment fluctuates around the neutral baseline, indicating that public emotional responses to fuel price and cost-of-living discussions vary considerably across different periods.

Compared with fuel price movements, sentiment trends appear substantially more volatile and less directly synchronised with short-term price fluctuations. While several periods of elevated fuel prices are associated with lower sentiment scores, the relationship is not consistently linear across the entire timeline. This suggests that online emotional responses are influenced not only by fuel prices themselves, but also by broader social, economic, and platform-specific factors.

A noticeable decline in sentiment can be observed during the sharp fuel price increase in early 2026, indicating stronger negative public reactions during periods of intensified economic pressure. However, sentiment values subsequently recovered as fuel prices stabilised, reflecting the dynamic and time-sensitive nature of online public opinion. Overall, the results show that sentiment changes are less directly aligned with fuel price movements than discussion volume, motivating the later correlation and lag-based analyses presented in this report.

2.4.5 Correlation Analysis

Table 1. Correlation analysis between fuel prices, discussion volume, and average social media sentiment.

Metric	Correlation
Fuel Price vs Discussion Volume	0.488
Fuel Price vs Average Sentiment	-0.082

Table 1 summarises the correlation analysis between official fuel prices, social media discussion volume, and aggregated sentiment scores. The correlation between fuel price and discussion volume is moderately positive (0.488), suggesting that periods of higher fuel prices are generally associated with increased online discussion activity related to fuel prices and cost-of-living concerns.

In contrast, the correlation between fuel price and average sentiment is relatively weak (-0.082). This indicates that although rising fuel prices may contribute to increased public attention, overall emotional responses on social media are likely influenced by a broader range of social, political, and economic factors beyond fuel price changes alone.

2.4.6 Cross-Platform Comparative Analysis

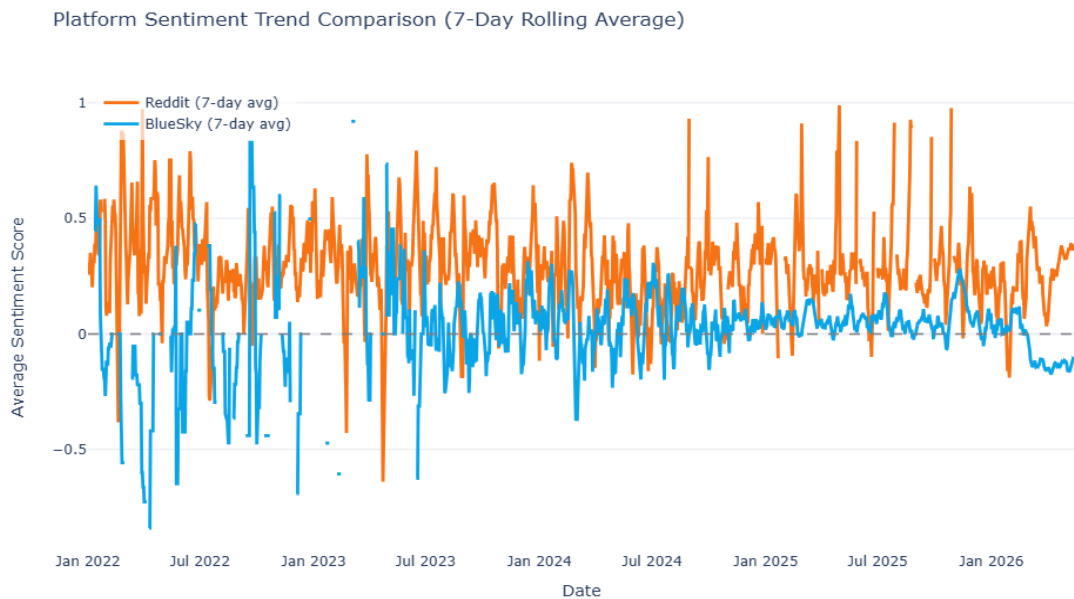


Figure 4. Cross-Platform Sentiment Dynamics Across Reddit and Bluesky

Figure 4 compares the rolling sentiment trends of Reddit and Bluesky over time. Reddit sentiment exhibits larger fluctuations throughout the observation period, while Bluesky sentiment remains comparatively closer to the neutral baseline for much of the timeline. Several periods of elevated Reddit sentiment variability can be observed during major fuel price fluctuations, suggesting stronger temporal changes in emotional expression within Reddit discussions. In contrast, Bluesky sentiment appears relatively more stable after 2024 despite substantial increases in posting activity.

Table 2. Platform-level comparison of average sentiment and total discussion volume.

Platform	Average Sentiment	Total Posts
Reddit	0.278	27151
Bluesky	0.029	45489

Table 2 shows the comparison between Reddit and Bluesky with respect to the number of posts and sentiment score of each platform based on the dataset obtained. It is clear that Bluesky generated a significantly higher number of posts compared to Reddit, mainly due to the fast development of the platform. On the other hand, Reddit had a significantly higher average sentiment score with regards to fuel price and cost of living-related discussions. Such discrepancies could be explained by the different users' characteristics of each platform. Specifically, Reddit posts usually contain longer threads of discussion, which facilitates detailed explanation and debate, and may lead to relatively higher sentiment scores. In contrast, Bluesky interactions tend to be short, thus generating relatively low sentiment scores despite the high post volume.

2.4.7 Lag Correlation Analysis

Table 3. Lag correlation between fuel prices and future social media sentiment across a 7-day offset window.

lag days	0	1	2	3	4	5	6	7
correlation	-0.082	-0.072	-0.073	-0.060	-0.063	-0.072	-0.082	-0.069

Table 3 displays the correlation of fuel prices with the future sentiment expressed through social media over a 7-day period lagged forward. The correlation coefficients are relatively low throughout all examined lag periods with

little variation from one period to another. The least negative correlation coefficient seems to be at a lag of 3-4 days, but all are close to zero.

This suggests that increases in fuel prices do not trigger a strong delayed shift in aggregated public sentiment within the following week. Compared with discussion volume, which exhibited a clearer relationship with fuel price fluctuations, sentiment appears substantially less directly coupled to short-term economic changes.

One possible explanation is that social media sentiment is inherently noisy and influenced by multiple overlapping factors, including political events, broader economic concerns, platform-specific discussion culture, and unrelated trending topics. As a result, although rising fuel prices may increase public attention and discussion activity, their effect on overall emotional polarity is less immediate and less consistent over time.

3. System Architecture and Design

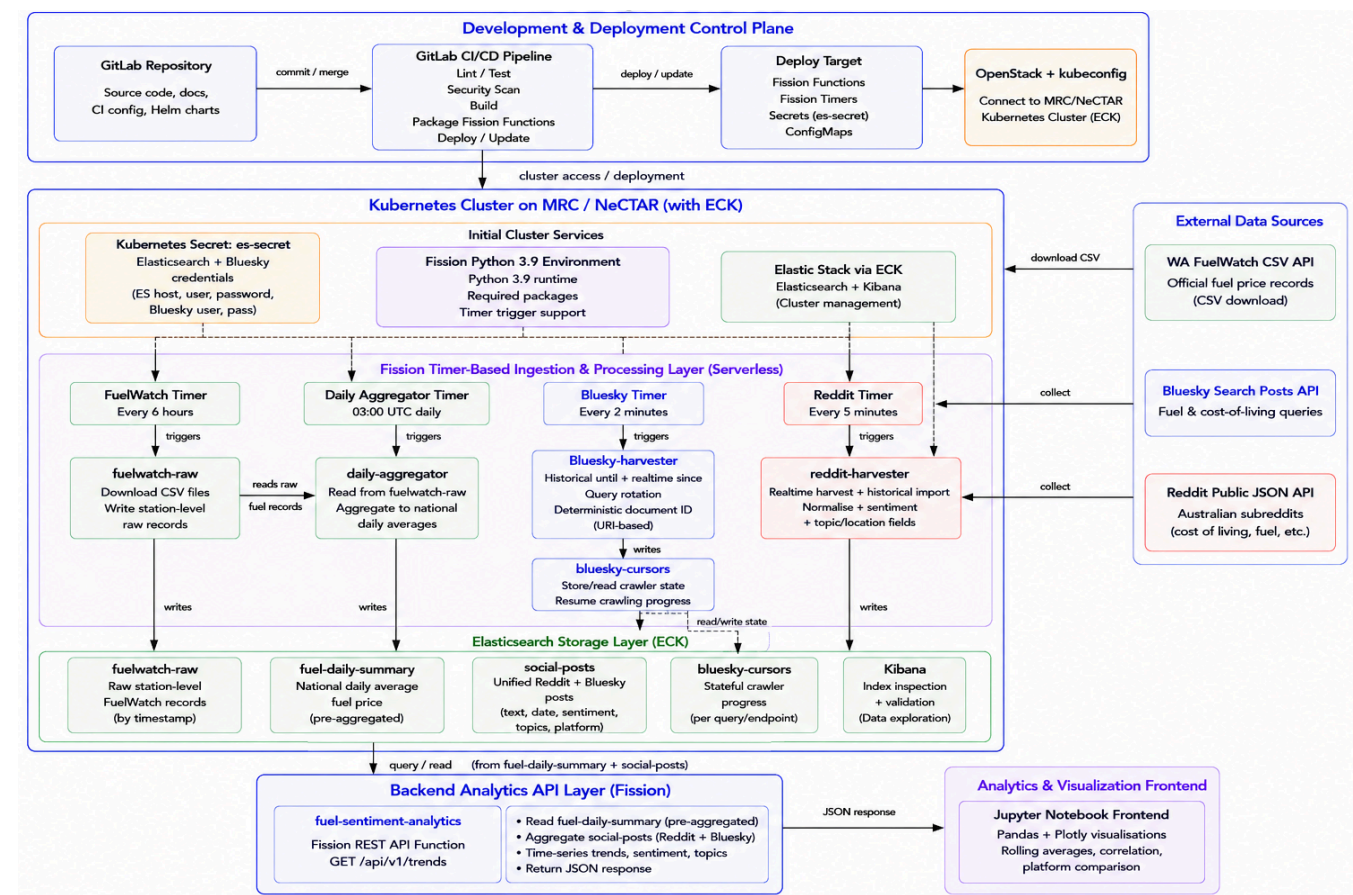


Figure 5. System architecture diagram

3.1 Overall System Architecture

The architecture consists of a five-stage pipeline which takes data from the external source and processes it through serverless ingestion, persistence, analytics API, and finally visualization in an interactive environment. The individual layers are separately deployable that lint, test, package the Fission functions, and configure the Kubernetes cluster deployed on the Melbourne Research Cloud (MRC) / NeCTAR environment through ECK operator.

3.1.1 External Data Sources

There are three publicly available external sources that will be used for the analyses. First is the Western Australian FuelWatch CSV API that delivers an official station level fuel price data in CSV files daily (Government of Western Australia, 2026). The second is the Bluesky Search Posts API, which is continually queried for posts containing information about fuel prices and cost of living sentiments on the Bluesky social platform. The third is

the Reddit Public JSON API scrapes posts and comments from Australian subreddits concerning the cost of living and fuel.

3.1.2 Fission Ingestion Layer

Data ingestion is handled by a set of serverless Fission functions running inside the Kubernetes cluster, a framework well-suited for modular, event-driven scientific data pipelines (Palade et al., 2019; Ríos-Monje et al., 2023). The ingestion layer consists of separate pipelines for FuelWatch, Reddit, and Bluesky data collection. Each pipeline is implemented as an independent Fission function responsible for data retrieval, preprocessing, and Elasticsearch integration.

The FuelWatch pipeline collects official fuel price records and produces aggregated daily summaries for downstream analytics. The Bluesky and Reddit pipelines continuously collect social media discussions related to fuel prices and cost-of-living topics, while also attaching metadata such as sentiment scores, topic labels, and platform information before storage. To support reliable ingestion, crawler state and intermediate processing metadata are maintained separately within Elasticsearch, allowing functions to resume collection safely and reduce duplicate processing across repeated executions.

3.1.3 Elasticsearch Data Storage Layer

All processed data is stored in an Elasticsearch cluster controlled by ECK. There are four indexes with different purposes: `fuelwatch-raw` stores timestamped station-level fuel price records; `fuel-daily-summary` holds pre-aggregated FuelWatch daily averages ready for rapid API queries; `social-posts` provides a unified store of Reddit and Bluesky content, with each document carrying text, date, platform, sentiment score, and topic fields; and `bluesky-cursors` maintains the crawler state required to resume collection without duplication. Kibana is deployed alongside Elasticsearch for index inspection, mapping validation, and exploratory data analysis during development.

3.1.4 RESTful API Layer

A Fission REST function named `fuel-sentiment-analytics` exposes the endpoint `GET /api/v1/trends`. At every invocation, it fetches pre-aggregated data from the `fuel-daily-summary` index and aggregates data from the `social-posts` index on the fly. Both aggregations take place independently in Elasticsearch by utilizing `size=0` queries. The result is returned in the form of a JSON response, enabling the Jupyter Notebook to get the analytics without interacting directly with Elasticsearch. It is important to mention that the connection between the function and Elasticsearch is established via the Kubernetes internal service URL.

3.1.5 Jupyter Notebook / Visualisation Layer

The analysis and presentation tier is a Jupyter Notebook environment that consumes JSON responses from the API layer. Using Pandas for data handling and Plotly for visualization to generate graphs depicting moving averages of fuel prices, correlation between price changes and sentiment, and comparisons of platforms between Reddit and Bluesky. The analysis and visualization layer is explicitly stateless when it comes to Elasticsearch; all data is received through the REST API.

3.2 Cloud Infrastructure

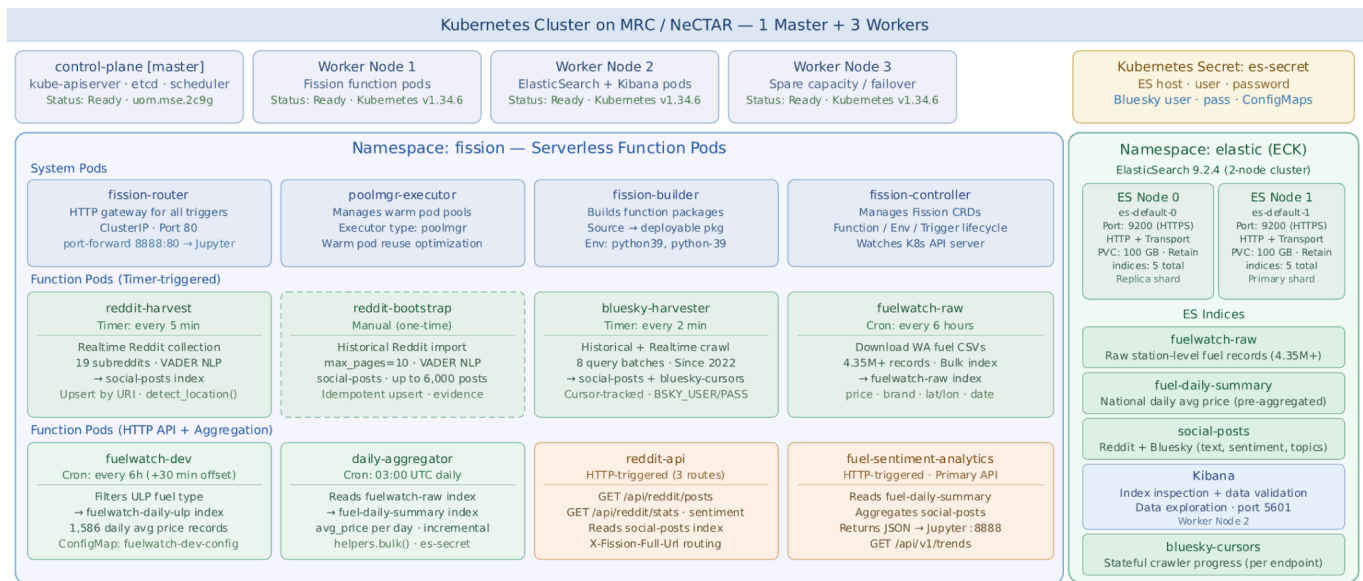


Figure 6. Kubernetes deployment diagram

3.2.1 MRC / NeCTAR Research Cloud

The system is deployed on an OpenStack-based research cloud provided through the NeCTAR Research Cloud programme in Melbourne, Australia. The infrastructure is hosted within the University of Melbourne project environment in the Melbourne region. Kubernetes access is configured through a kubeconfig file used to authenticate with the Kubernetes API server for cluster operations. The cluster runs Kubernetes v1.34.6 on this infrastructure. Elastic Cloud on Kubernetes (ECK) is used to manage Elasticsearch and Kibana resources through Kubernetes-native custom resources and an operator-based architecture.

3.2.2 Cluster Topology and Kubernetes Nodes

The cluster consists of one control plane node and three worker nodes, all in Ready state. The control plane node provides Kubernetes control plane functionality, including API server, scheduler, controller manager, and etcd for cluster state management. The worker nodes execute application workloads, including Fission serverless functions and stateful services such as Elasticsearch and Kibana. Workloads are distributed across worker nodes to improve resource utilisation and provide resilience against node failure or maintenance events.

3.2.3 Namespaces

The cluster workloads are organised across multiple Kubernetes namespaces, each serving distinct system and application roles. The Fission namespace contains part of the server-side components and supporting roles, while most Fission control-plane components and RBAC bindings are actually deployed in the default namespace. The default namespace hosts the majority of Fission components, including executor, builder manager, router, storage service, kubewatcher, timer, webhook, and KEDA-based trigger services, and also stores configuration resources such as ConfigMaps and Secrets, which are injected into components via Kubernetes volume mounts. In addition, dynamically generated function runtime pods (e.g., poolmgr-based Python execution pods) are scheduled in the default namespace. The elastic namespace is managed by the Elastic Cloud on Kubernetes (ECK) operator and contains a two-node Elasticsearch cluster, a single Kibana instance, and the operator itself. Other system namespaces such as kube-system, calico-system, and keda provide infrastructure-level services. Cluster-scoped resources like StorageClass are managed at the cluster level rather than within individual namespaces.

3.2.4 Persistent Storage

Persistent block storage is provisioned through a Kubernetes StorageClass named perffretain, which uses the cinder.csi.openstack.org CSI driver backed by OpenStack Cinder. The StorageClass is configured with a volume

type of performance and a Retain reclaim policy, ensuring that persistent volumes are preserved after their associated PersistentVolumeClaims are released. The Elasticsearch deployment uses two 100Gi PersistentVolumeClaims, providing approximately 200Gi of durable storage for index data. These volumes are dynamically provisioned and attached to Elasticsearch data nodes managed by the Elastic Cloud on Kubernetes operator. In addition, Fission uses an 8Gi persistent volume for internal storage to support state persistence across pod restarts.

3.2.5 Elastic Cloud on Kubernetes

Elasticsearch and Kibana are deployed using Elastic Cloud on Kubernetes, which runs as an operator pod in the elastic namespace. Through custom resource definitions (CRDs), the operator extends the Kubernetes API and provides declarative management for Elasticsearch clusters and Kibana. The operator constantly ensures that the desired and current states of the resources are aligned. The Elasticsearch deployment comprises two nodes, each having its own persistent storage volume. The Elasticsearch cluster can be accessed via a Kubernetes Service exposing the Elasticsearch HTTP API on port 9200 and internal transport protocol on port 9300. Kibana version 9.2.4 runs as a single replica in the same namespace and connects to the Elasticsearch cluster for data exploration and visualization.

3.3 Fission Data Ingestion Design

The data collection layer of this system is implemented based on the Fission serverless framework deployed on Kubernetes. Different data sources are respectively handled by independent Fission functions for collection, and the data is written into the corresponding ingestion indexes in Elasticsearch. The subsequent Elasticsearch optimisation, aggregation, and API-facing data preparation will be discussed in the next section. All pipelines run in a unified Python 3.9 Fission environment and are managed using the package, function, and trigger separation method. Functions read Elasticsearch connection details and API credentials during runtime through Kubernetes Secrets. This modular design enables the Reddit, Bluesky, and FuelWatch pipelines to be independently developed and updated without affecting other data collection processes.

The two social media platforms, Reddit and Bluesky, adopt a unified `social-posts` schema during the ingestion stage. They are distinguished by the `platform` field. Common fields include `platform`, `text`, `created\_at`, `date`, sentiment fields, topic indicators, etc. The purpose of this design is to enable different platforms' data to use the same fields for filtering, aggregation, and platform-level comparison in subsequent Elasticsearch processing, API-level analytics, and Notebook analysis. Both social media pipelines use platform-specific unique identifiers as Elasticsearch document IDs, allowing duplicate collected posts to update existing documents instead of generating duplicate records.

Bluesky Ingestion Pipeline

In designing the Bluesky ingestion pipeline, we take advantage of the lightweight execution framework offered by the Fission environment that uses a timer-triggered approach. The Bluesky ingestion pipeline involves calling the Bluesky Search Posts API per execution, retrieving a few posts about Australian fuel prices and the cost of living on Bluesky, preprocessing, and storing the preprocessed document to Elasticsearch.

For supporting historical crawling as well as real-time crawling, the pipeline uses an incremental approach to historical crawling as well as real-time crawling. The process of historical crawling will be used for retrieving historical data, whereas real-time crawling will be used for retrieving new posts. This design reduces the workload of individual executions and improves the stability of timer-triggered ingestion within the Fission environment.

Due to the statelessness of the Fission functions, the crawler state is persisted in the Elasticsearch `bluesky-cursors` index instead of in-memory storage. Crawler state containing information about the crawler state that needs to be considered during subsequent crawls is stored. This enables the pipeline to continue collecting data incrementally through multiple executions triggered by the timer.

Collected Bluesky posts are normalised into the shared `social-posts` schema used by the social media ingestion layer. Common fields include platform, text, timestamps, sentiment fields, topic indicators, and location-related fields. Bluesky post URLs serve as Elasticsearch document IDs to ensure consistency of documents on consecutive ingestion runs.

Reddit Ingestion Pipeline

The Reddit ingestion pipeline is responsible for gathering data pertaining to fuel price discussions and cost of living matters from specific subreddits related to Australia. This ingestion pipeline consists of multiple Fission functions that have been allocated various tasks related to real-time collection, historical backfill, and accessing APIs. The `reddit-harvest` Fission function is responsible for real-time collection of data in an incremental fashion via timers from Reddit public JSON endpoints. Historical backfill is achieved via the `reddit-bootstrap` workflow.

Collected Reddit posts are normalised into the shared `social-posts` schema together with Bluesky data. Additional preprocessing steps generate sentiment fields, topic indicators, and location-related metadata to support downstream aggregation and analytical workflows. Reddit records are distinguished from other social media data through the `platform` field while still maintaining a unified schema structure for Elasticsearch processing and cross-platform comparison. The Reddit pipeline also includes HTTP-triggered query functions used by downstream analytical workflows and API-level access.

FuelWatch Ingestion Pipeline

The FuelWatch ingestion pipeline is responsible for collecting officially published fuel price records from the FuelWatch service in Western Australia. Records include fuel prices at the gas station level, fuel type, release date, and gas station metadata extracted from historical CSV reports. Since FuelWatch provides structured official economic data rather than social media content, these records are stored in a dedicated Elasticsearch index, separate from the shared `social-posts` index.

During ingestion, the Fission function periodically downloads the official FuelWatch CSV datasets and writes normalized station-level fuel records into the `fuelwatch-raw` index. In order to improve the stability during large-scale historical data import, we employ batch indexing and controlled file processing strategies. We intentionally kept the raw records because it would facilitate reaggregation and recomputation at a later date without having to repeat the download of the full historical data set from an external data source.

During development, we introduced a lightweight preprocessing and verification workflow called fuelwatch-dev. This workflow generates a lightweight Elasticsearch validation index called fuelwatch-daily-ulp. This index stores the preaggregated daily ULP average price. This helps us to validate the downstream aggregation results generated from the raw fuelwatch-raw records.

By comparing Elasticsearch aggregation outputs against the precomputed daily ULP averages stored in fuelwatch-daily-ulp index, the development process was able to verify the correctness and consistency of backend aggregation logic without repeatedly reprocessing the complete raw dataset.

Since the fuelwatch-daily-ulp validation index contained only a small number of development-oriented validation records and was not involved in downstream production analytics or API workflows. We intentionally omit sharding to avoid unnecessary storage and management overhead.

In the final production workflow, the official fuelwatch records are mainly ingested and saved in the fuelwatch-raw index. Downstream aggregate outputs are then stored in the fuel-daily-summary index for API-level analysis and frontend visualization. Therefore, the primary responsibility of the FuelWatch ingestion pipeline is to provide reliable ingestion and persistent storage of official fuel price data for Elasticsearch, establishing a stable foundation for downstream aggregation, analytical processing, and visualization workflows.

3.4 Elasticsearch Storage Design

Our system maintains four indices in the Elasticsearch cluster, as well as a lightweight internal validation index used during development. As the volume of data has grown during development, the field mappings and shard settings have been progressively optimized and adjusted.

Index Overview

Table 4. Overview of Elasticsearch indices and their roles

Index	Alias	Shards	Documents	Role
social-posts-v1	social-posts	3	70,000+	Harvested social media posts
fuelwatch-raw-v1	fuelwatch-raw	3	4,000,000+	Raw station-level fuel price records
fuel-daily-summary	—	3	1500+	Pre-aggregated daily average fuel prices
bluesky-cursors	—	1	~59	Bluesky crawler pagination state
fuelwatch-daily-ulp (Internal Validation)	—	1	1500+	Internal validation dataset for aggregation verification

social-posts-v1 (alias: social-posts)

This index stores all the posts harvested from Reddit and Bluesky. Sentiment scoring is performed using VADER (Hutto & Gilbert, 2014) during data import, so each document arrives with a `'sentiment_score'` field. Key fields include `created_at` (date, UTC), `platform` (keyword: "bluesky" or "reddit"), `text` (full-text), `sentiment_score` (float, from -1.0 to $+1.0$), `sentiment_label` (keyword), and topic indicator booleans `is_au`, `is_fuel`, and `is_cost`. Since social media posts do not contain explicit geolocation data, location information is stored as `'matched_location'` (keyword), a value inferred from the post content with city-level accuracy. Reddit posts also include a `'subreddit'` field, which is stored as a multi-field type to support both full-text search and keyword aggregation.

fuelwatch-raw-v1 (alias: fuelwatch-raw)

The `fuelwatch-raw-v1` index stores raw station-level price records from the FuelWatch WA API. It currently contains around 4.37 million records, about 1.26 GB of data, and is split across three shards so aggregations can run in parallel across the two cluster nodes.

fuel-daily-summary

The API should not aggregate 4.37 million raw records for every request. That would be too slow for a real-time endpoint. To avoid this, the `'fuel-daily-summary'` index stores a daily summary instead. Each row contains only the date and the average FuelWatch price in cents per litre. This reduces the raw dataset to about 1,596 daily records. The `'daily-aggregator'` Fission function refreshes the index every night at 03:00 UTC.

bluesky-cursors

The `'bluesky-cursors'` index is only used by the Bluesky harvester. It saves the crawler's progress between timer-triggered runs. The index stores last-seen timestamps, historical `'until'` cursors, query batch positions, and markers for completed crawls. This lets the harvester continue from where it stopped, instead of starting again each time. It is not accessed by the API layer or Jupyter frontend.

fuelwatch-daily-ulp

The `fuelwatch-daily-ulp` index was created during development and testing. It was a small development index used for us to check that the daily aggregation results were correct before the data was used in later analysis.

Sharding and Alias Strategy

Both social-posts-v1 and fuelwatch-raw-v1 were first created with one shard. As the datasets became larger, both indices were moved to a three-shard setup. New indices were created, and the data was copied across using `_reindex`. After that, the document counts were checked. Once the counts matched, the old indices were deleted.

Aliases were then added to the new indices. The aliases were social-posts and fuelwatch-raw. Because the harvesters wrote to these aliases, no code changes were needed after the migration.

Timezone Handling

All `date_histogram` aggregations use `time_zone` such as: "Australia/Melbourne". This makes sure records are grouped by the correct local calendar date. Without this setting, posts created after 14:00 AEST could be placed into the next UTC day. That would create a 10-hour mismatch between fuel price dates and social post dates.

3.5 API Layer and Visualisation

The backend provides one RESTful endpoint: `GET /api/v1/trends`. It is implemented as a Fission serverless function called `fuel-sentiment-analytics` and runs on the MRC cluster. The function connects to Elasticsearch through the internal Kubernetes service URL, `elasticsearch-es-http.elastic.svc.cluster.local`. This works because both services run inside the same cluster.

The backend does not fetch raw records and process them manually. Instead, it sends the aggregation work to Elasticsearch. Fuel price data is read from `fuel-daily-summary`. Social sentiment data is aggregated from `social-posts-v1` in real time, using a daily `date_histogram`.

After this, the two datasets are joined in memory by date. The API returns the result as a time-sorted JSON array. Each item follows the same structure. It includes `date`, `national_avg_price`, `total_posts`, `overall_avg_sentiment`, and a platform object. The platform object stores sentiment values and post counts for Reddit and Bluesky.

The Jupyter Notebook uses this endpoint through a normal HTTP request. It does not query Elasticsearch directly. After receiving the JSON response, it loads the data into Pandas DataFrames. It then creates interactive Plotly charts, summary statistics, correlation analysis, and scenario-based trend analysis.

This design keeps the Notebook separate from Elasticsearch. It also means the backend aggregation logic can change without changing the frontend workflow.

4. Implementation and Deployment

4.1 Team Contributions

Table 5. Team member contributions and responsibilities

Xinyao Li	Designed and deployed the Bluesky Fission ingestion pipeline, implemented Elasticsearch integration and produced system architecture and deployment diagrams.
Hanyue Li	Designed and deployed the Elasticsearch cluster infrastructure, implemented the distributed storage architecture, and developed the backend REST API and daily aggregation pipeline as Fission serverless functions.
Junyao Zhang	Initial Kubernetes cluster setup, kubeconfig sharing, Reddit Fission pipeline, Reddit realtime and historical ingestion. Produced deployment diagrams.
Yichen Sun	Designed and implemented the CI/CD pipeline infrastructure on GitLab. Managed release coordination and ensured consistent deployment across the team's development lifecycle.
Hongkun Zhang	Designed, implemented, and deployed the FuelWatch data pipeline, developed Jupyter Notebook frontend visualisations, and conducted data analysis.

4.2 Repository Structure and Development Workflow

The GitLab repository has five main folders. The backend folder contains the Fission function code and deployment specs for Reddit, Bluesky, FuelWatch, and the analytics API. The frontend folder contains the Jupyter Notebook and visualisation scripts. The database folder contains Elasticsearch mappings and the ECK deployment files. The test folder contains unit tests for crawler logic, cursor handling, and API response structure. The docs folder contains extra deployment notes. The README file at the repository root explains environment setup, secret configuration, and deployment steps.

For development, each team member worked on a separate branch for their own component. These components included ingestion pipelines, storage, CI/CD, and frontend analysis. Changes were submitted through merge requests before being merged into main. GitLab issues were used to record bugs and deployment tasks. The CI/CD pipeline ran on merge requests and checked linting, dependency scanning, and unit tests. These checks helped catch some basic problems before deployment, but the final validation still had to be done manually on the live Kubernetes and Elasticsearch environment.

4.3 Deployment Process

4.3.1 Cluster Connection and Base Services

The deployment process starts by connecting to the team's shared Kubernetes cluster. After the cluster is configured, each team member sets up local Kubernetes access with the provided kubeconfig file.

Once the connection is ready, we check the cluster nodes, namespaces, and core services. This confirms that the basic deployment environment is working correctly. It also ensures that the functions deployed later can access the required cluster services.

The Elastic Stack is deployed on the Kubernetes cluster using the Elastic Cloud on Kubernetes, or ECK, operator. The ECK operator is installed with Helm. Before deploying Elasticsearch, we configure a storage class for persistent volumes. This prevents Elasticsearch data from being lost when pods restart.

Elasticsearch and Kibana are then deployed in the elastic namespace. They communicate through the internal Kubernetes service network. After deployment, we check the pod status to confirm that both Elasticsearch and Kibana are running.

During deployment, Kibana is accessed through port forwarding. This allows us to check indices and sample records before the system is fully connected.

4.3.2 Fission Runtime and Shared Configuration

Before deploying the specific ingestion functions, the team first created a shared Python 3.9 Fission environment. Python functions related to Reddit, Bluesky, FuelWatch, and backend analytics all run based on this environment. The deployment method for each function is basically the same: first, package the Python source files and requirements.txt into a zip archive, then create or update the Fission package, and finally create or update the corresponding Fission function based on the package.

Elasticsearch connection details and API credentials are provided through Kubernetes Secret es-secret instead of being directly written into the source code. For different data sources, if there are additional credentials, they will also be included in the corresponding deployment configuration. For example, the Bluesky harvester, in addition to Elasticsearch credentials, also requires Bluesky API credentials. In general, when deploying Fission functions, it is necessary to configure the package, function endpoint, secret or ConfigMap, as well as the corresponding timer trigger or HTTP trigger.

4.3.3 Deployment of Dataset Ingestion Function

Reddit Fission Deployment

The deployment of the Reddit pipeline consists of three main Fission functions and one local script: `reddit-harvest`, `reddit-bootstrap`, `reddit-api`, and `reddit_history_import.py`. `reddit-harvest` was attached to a timer trigger for scheduled execution within the cluster, with the timer running every five minutes. `reddit-bootstrap` is used for manual historical backfill and is mainly used during the initial data import phase. `reddit_history_import.py` is a local Python script that fetches historical data via the Arctic Shift API and bulk-imports matching posts into Elasticsearch directly. `reddit-api` exposes query interfaces through HTTP triggers and is used for subsequent access to Reddit posts, statistics, and sentiment results. These Reddit functions use a shared Python 3.9 Fission environment and connect to Elasticsearch through the shared Elasticsearch secret configuration. After deployment, the processed Reddit records are written to the shared `social-posts` index.

Bluesky Fission Deployment

The deployment of the Bluesky pipeline begins with the packaging of crawler files. The deployment package includes `bluesky_harvester.py`, `bluesky_processor.py`, `bluesky_storager.py`, and `requirements.txt`, and is packaged as `bluesky-harvester.zip`. Subsequently, this zip archive is uploaded and created as the Fission package `bluesky-pkg`. After confirming that the package status changes to `succeed`, the Fission function `bluesky-harvester` is created, with the entrypoint set to `bluesky_harvester.main`.

This function mounts `es-secret`, which is used to read Elasticsearch connection details and Bluesky credentials during runtime, including `BSKY_USER` and `BSKY_PASS`. The function timeout is set to 120 seconds. The execution is automatically triggered through the timer trigger `bluesky-timer`, with the schedule set to `@every 2m`. To improve deployment reproducibility, the configurations of the Bluesky package, function, and timer trigger are also saved as Fission specs, making it convenient to re-deploy based on the same configuration in the future.

FuelWatch Fission Deployment

The deployment of the FuelWatch pipeline is primarily responsible for periodically collecting official FuelWatch CSV data and writing fuel price records to Elasticsearch. The FuelWatch source code and dependencies are packaged as Fission packages and deployed on the shared Python 3.9 runtime.

The raw ingestion function, `fuelwatch_raw_ingestion`, is responsible for downloading station-level FuelWatch records and writing them directly to the `fuelwatch-raw` index in Elasticsearch. This raw ingestion stage is separate from the aggregation stage, which processes the raw records into daily summaries and is discussed in Section 4.3.4. The FuelWatch data pipeline runs automatically through timer triggers configured by cron expressions. There is no need to manually perform data ingestion and preprocessing tasks. The FuelWatch data ingestion and preprocessing workflow employs a `0 */6 * * *` schedule that executes every six hours. This improves the system's ability to cope with delays in the release of upstream FuelWatch data.

4.3.4 Backend Analytics API Deployment

The backend analytics layer is deployed with Fission serverless functions. There are two functions: `daily-aggregator` and `fuel-sentiment-analytics`. Both are packaged as ZIP files and deployed with fission fn create using the `python-39` environment. The `es-secret` is mounted into each function so it can access Elasticsearch credentials at runtime. The `daily-aggregator` function processes the previous day's raw fuel records from `fuelwatch-raw`. It writes the daily average results into the `fuel-daily-summary` index. The analytics API then reads from this summary index when handling requests.

This function runs on a timer trigger called `fuel-daily-aggregator`. The trigger uses the cron expression `0 3 * * *`, so the summary index is refreshed every night at 03:00 UTC. The `fuel-sentiment-analytics` function is exposed through an HTTP route at `/api/v1/trends`. It queries Elasticsearch, combines fuel price data with sentiment data, and returns the result as JSON. The Jupyter Notebook frontend then uses this JSON response for analysis and visualisation.

4.3.5 Deployment Acceptance Check

After deployment, a series of basic acceptance checks were performed to verify that the ingestion, storage, and API layers were functioning correctly. Fission functions were manually triggered to confirm successful execution, while Elasticsearch indices and sample records were inspected through Kibana and curl queries after port-forwarding. REST API endpoints were also tested to ensure valid JSON responses could be returned to the frontend Notebook. These checks were primarily used to validate successful deployment and service connectivity. More comprehensive testing and validation procedures are discussed separately in Section 8.

5. Scalability

5.1 Modular Pipeline Scalability

The system was designed using modular Fission-based pipelines, allowing new data sources and preprocessing workflows to be added independently without modifying existing components. For example, the FuelWatch pipeline was extended during development through an additional internal preprocessing workflow used for intermediate testing and validation. The extension was deployed through separate Fission functions and timer triggers while reusing the existing Elasticsearch deployment environment. To integrate a new harvester, only a new Fission package, function, timer trigger, and Elasticsearch index configuration are required. Existing pipelines can continue operating without major structural changes.

5.2 Fission Function Scalability

Functions for fission were used as independent serverless components, which enabled both the ingestion and preprocessing to take place independently using timers. This minimized the coupling that existed among various stages within the pipeline. In the case of social media harvesters, the frequency of the timers as well as the amount of work done on each execution can be configured separately from the FuelWatch pipeline. So Bluesky and Reddit ingestion can be adjusted without changing the FuelWatch pipeline.

The timing of the process scheduling and frequency of processing can be tuned in real time based on data availability and ingestion processing load. In the case of FuelWatch, the initial setup was to schedule the processing task once per day. After observing that the FuelWatch data publication would occur later than expected, and the result of processing was all zeros due to a lack of upstream data, the processing frequency was tuned to six hours. This improved pipeline reliability and reduced the impact of incomplete upstream data without requiring manual intervention.

Serverless deployment architecture helped update or redeploy each function independently without affecting other parts of the pipeline. The use of a queue design using Redis or KEDA can improve performance for heavy loads, but it was not implemented in the final system due to project scope and resource constraints.

5.3 Elasticsearch Scalability

The high-volume indices, `fuelwatch-raw-v1` and `social-posts-v1`, are configured with three primary shards and one replica. Smaller indices, such as `bluesky-cursors`, use fewer shards because they store much less data.

For `fuelwatch-raw-v1`, the three-shard setup spreads 4.37 million records across both Elasticsearch data nodes. This allows aggregation queries to run in parallel instead of being handled by one node only. During verification, each shard stored about 1.45 million records and was around 215 MB in size. The primary shards were balanced across `elasticsearch-es-default-0` and `elasticsearch-es-default-1`. Replica shards add another layer of reliability. If one node becomes unavailable, the replica can still provide access to the data. Replicas can also help serve read requests when query load is high.

The system also separates different types of data into different indices. Raw records, daily summaries, and crawler state are stored separately. This makes the system easier to scale and manage. The `fuel-daily-summary` index works like a materialised view. It reduces 4.37 million raw fuel records to about 1,596 daily rows. API queries read from

this summary index directly. This keeps response times low, even as more raw records are added. Without this summary layer, the API would need to aggregate the full fuelwatch-raw-v1 index on every request, which would become more expensive over time.

The alias strategy also supports future scaling. Harvesters and API functions use aliases instead of physical index names. This means the underlying indices can be replaced or restructured without changing application code. If the data grows significantly, more shards can be added by repeating the `_reindex` and alias migration process. Running services do not need to be changed, because they continue to use the same aliases.

At the cluster level, Elasticsearch runs on two nodes. Primary and replica shards are distributed across separate nodes. This improves read capacity and provides room for future expansion. If more nodes are added later, Elasticsearch can rebalance shards across the cluster automatically. The existing alias and shard setup does not need to change, because shard rebalancing is handled by Elasticsearch itself.

5.4 API and Frontend Scalability

On the frontend side, the Jupyter Notebook consumed aggregated API responses rather than directly querying raw Elasticsearch records, reducing client-side processing overhead when handling larger datasets and longer time ranges.

6. Discussion Strengths and Weaknesses of Technologies

In this section, the key technologies adopted in the project are evaluated from a practical standpoint, with reflection on their suitability for a real-world cloud-based data pipeline and the trade-offs encountered during development and deployment.

6.1 MRC / NeCTAR Research Cloud

Melbourne Research Cloud (MRC), part of the Australian NeCTAR consortium, is an OpenStack-based academic cloud computing environment and was the core of the cloud computing infrastructure utilized in this project. It enabled the team to deploy a multi-node Kubernetes cluster on genuine cloud hardware, with access controlled through OpenStack identity credentials and a shared kubeconfig. Persistent block storage was provisioned through the Cinder CSI driver, supplying durable volumes for Elasticsearch data nodes under a Retain reclaim policy.

The most significant advantage of NeCTAR is that it provides a true cloud environment as opposed to a virtual one. The team gained direct exposure to authentic infrastructure concerns: multi-node pod scheduling, persistent volume lifecycle management, and network-level service isolation. The academic allocation model removed financial pressure, allowing extended iteration without cost constraints typical of commercial clouds. Identity management in OpenStack being centralized helped to simplify the sharing of access to the cluster between the five team members using one single kubeconfig file. Kubernetes cluster provisioning through Magnum was well-integrated into the NeCTAR environment and produced a stable, accessible cluster throughout the project.

The primary drawback was the complexity of initial setup. Configuring the OpenStack CLI toolchain, including sourcing the RC file, installing the Magnum plugin, and aligning client library versions with the correct Python interpreter introduced friction that delayed onboarding for some team members. Network accessibility presented a recurring challenge: all cluster services are internal by default, requiring active `kubectl port-forward` sessions that expire when the terminal closes. This absence of stable public endpoints meant that collaborative access to running services required constant coordination. Additionally, the academic quota model restricted the cluster to a single control-plane node, making the Kubernetes API server itself a structural single point of failure. While worker-node workloads tolerate transient control-plane unavailability, any deployment or configuration change is blocked until the control plane recovers, which is a meaningful constraint for a continuously ingesting system.

6.2 Kubernetes

Kubernetes served as the standard container orchestration layer (Carrión, 2022) for all project workloads, including the Fission serverless framework, the Elasticsearch cluster managed by ECK, and the Kibana dashboard. All resources were expressed as declarative YAML manifests committed to the GitLab repository.

Kubernetes delivered strong operational value through three main mechanisms. First, namespace separation helped isolate major infrastructure components. The elastic namespace had Elasticsearch, Kibana, and ECK operator, whereas the Fission namespace and its related runtime had the other services. Second, automatic pod rescheduling and workload recovery (Mondal et al., 2024) improved the resilience of the continuous ingestion pipeline. If any harvesting pod crashes, the poolmgr executor can easily replace the pod using the pre-warmed pool without any manual intervention, whereas the ECK operator will recover the Elasticsearch pods and attach the persistent volume to the pod. Third, Kubernetes used a declarative configuration model, with all manifests version-controlled in GitLab. This made it easier to recreate cluster resources after accidental deletion or deployment issues. Pod-level logging through kubectl also helped accelerate debugging by exposing runtime errors more clearly.

Kubernetes also introduced considerable operational complexity. Version alignment across multiple components (Kubernetes release, the Fission CLI, the ECK operator, and third-party Helm charts) required careful attention; mismatches between tool versions caused subtle failures that were difficult to diagnose without cross-referencing changelogs. Secret management was another pain point: Kubernetes Secrets carried no built-in schema validation, meaning an incomplete secret update silently broke all dependent functions until the omission was traced through logs. More broadly, the Kubernetes and Fission setup introduced several layers where deployment problems could occur. Issues could come from package building, Python environments, pod startup, HTTP routing, or Elasticsearch connections. In many cases, these problems only appeared as a generic 500 error, so debugging often required checking Fission logs, Kubernetes pods, and Elasticsearch results together. For a small team, working across kubectl, the Fission CLI, Helm, and ECK also increased the overall deployment and debugging complexity.

6.3 Fission Pros and Cons

Fission acts as the main means of implementing the data harvesting jobs in the Kubernetes cluster. This technique fits perfectly for the task due to its ability to perform tasks occasionally. Harvesting data from the three sources mentioned does not require a background crawler to be kept running all the time, but it can be scheduled. Fission timer triggers fit this workload well, enabling the collection tasks to run automatically within the Kubernetes cluster without relying on locally running scripts or manual execution.

Another advantage of Fission is the clear separation between ingestion pipelines. Different data sources can be split into independent functions, so each pipeline can be developed, deployed, and updated independently. This separation reduced coupling between different data sources because modifying a crawler for one source would not directly affect other pipelines. The package, function, and trigger of Fission also manage code dependencies, execution entry points, and scheduling methods separately, making deployment and maintenance easier to manage.

Fission can also be well integrated with Kubernetes' configuration management. The connection details for Elasticsearch and API keys could be passed into the function using a Kubernetes secret instead of having them hardcoded in the function. This is not only a safer method than having the information in a script but is also more suitable for use in a shared cluster environment.

But Fission does not just run the Python script on the cloud. Just because a script is run successfully locally, this does not mean that the same script will be successfully executed if packaged in a Fission function. There are multiple aspects, such as the Python environment, version of dependencies, packaging itself, function entry point, Kubernetes secrets, and the Elasticsearch connection, which can become the cause of unsuccessful deployment. When trying to solve problems, it is usually necessary to simultaneously check Fission logs, Kubernetes pod status, and Elasticsearch write results. This is more complex and time-consuming than simply debugging the local Python script directly.

Fission also has limitations for the harvester logic. Data harvesters often need to process historical data, save progress, and may run for a long time, while the Fission function is more suitable for short-term, clearly defined tasks. If too much crawling workload is placed in a single execution, the function can easily hit timeout and resource limits. Therefore, Fission is not suitable for completing large-scale historical crawling in one go; it is more suitable for incremental ingestion, where collection tasks are completed gradually across multiple executions.

Another drawback is that the Fission function itself does not save the running state. After a function execution finishes, local variables and memory states will disappear, so the crawler cannot rely on the local state to remember where the last collection was. For data sources that require continuous incremental collection, the state must be saved in an external system, such as Elasticsearch. This design increases the implementation complexity, but also makes the pipeline easier to recover after a function restart or pod recreation.

Overall, Fission worked well for incremental timer-triggered ingestion workloads. Compared to running crawlers locally, Fission introduced additional limitations such as restrictions on the time taken by functions, their state, and the overall workload. This meant that data extraction operations had to be split into smaller chunks, and functions had to remain lightweight and short-running to operate reliably within the serverless environment.

6.4 Elasticsearch and Kibana

Kibana: User Interface and Data Visibility

Kibana can feel overwhelming at first. The interface has many options, and the default view is not always easy to read. However, it becomes much more useful once important fields such as `'platform'`, `'created_at'`, and `'sentiment_score'` are pinned to the active view.

After this setup, raw documents can be viewed as clear time-series dashboards without using extra tools. Kibana was also useful for system auditing. For example, when the social media harvesters stopped collecting data, we checked the `'ingested_at'` timestamps in Kibana. This helped us find the exact time when the pipeline went offline.

The Dev Tools console was another important feature. It allowed us to write and test Elasticsearch queries directly in the browser. We could also optimize the queries there before moving them into the Python backend.

However, Kibana has a clear learning curve. In a relational database, data is usually shown in structured tables. In Kibana, the default Discover screen shows records as raw JSON documents. This makes the data harder to read at a glance unless the view has already been configured.

Developers also need to understand indices, create a Data View, and apply filters before they can query the data effectively. This workflow is less intuitive for users who are more familiar with standard SQL tools.

Elasticsearch: Storage Layer

Elasticsearch handled the project's scale well. The system stored more than 4.35 million raw fuel records. Instead of sending all of this data to the backend, the aggregation work was pushed down to the three primary shards. As a result, the Python backend only received about 1,500 daily summary rows. This kept memory usage low enough for the Fission container limits. The index alias feature was also useful during shard migration. For example, the `social-posts` alias was pointed to `social-posts-v1`. Because the harvesters and API functions used the alias, they continued to run without any code changes.

However, Elasticsearch also introduced some challenges. It is a NoSQL distributed search engine, so it does not support strict relational joins like a traditional SQL database. The ES|QL query language was also harder to write than standard SQL. The Kibana “Translate to Query DSL” tool helped reduce this difficulty by converting ES|QL-style queries into Elasticsearch Query DSL.

At first, we tried to connect fuel data and social media posts using database-level lookups and ENRICH policies. This did not work as expected. Elasticsearch could only match the first document it found for each key, instead of

calculating a true aggregate. Because of this, we changed the design. Both datasets are now aggregated separately inside Elasticsearch. The final merge is then done in the application layer.

7. Error Handling and Limitations

7.1 Deployment and Runtime Issues

7.1.1 Cloud and Cluster Tooling Issues

OpenStack CLI Missing COE Extension

During cluster management setup, the openstack coe commands initially failed because the Container Orchestration Engine (COE) extension was unavailable. Investigation showed that the issue was caused by a Python environment mismatch: the openstack CLI binary was running under Python 3.9, while the python magnumclient package had been installed into a separate Python 3.14 environment. Since OpenStack client plugins are interpreter specific, the CLI could not detect the Magnum extension despite the package being installed elsewhere on the system.

The issue was resolved by reinstalling python magnumclient into the same Python 3.9 environment used by the openstack command. After the package was installed under the correct interpreter, the openstack coe cluster list and related cluster management commands functioned correctly.

Fission Client Server Version Mismatch

When the Fission CLI was installed via Homebrew, the package manager resolved to version v1.21.0, which did not match the Fission server version v1.22.0 already deployed on the Kubernetes cluster. The Homebrew-managed version could not be easily pinned to a specific release without formula modifications. The resolution was to bypass Homebrew entirely and install the CLI manually by downloading the exact v1.22.0 binary directly from the official Fission GitHub releases page. The correct binary was fetched using curl, granted executable permission with chmod +x, and moved to /usr/local/bin/ to replace the mismatched version. After this, the fission version confirmed that both the client and server reported v1.22.0, and all subsequent CLI operations functioned correctly.

Redis Installation Failure

The initial Helm-based Redis installation failed due to a version incompatibility between the Helm chart (v0.16.9) and the Redis operator (v0.23.0). The newer chart version stopped generating the redisConfig field in the Custom Resource manifest, producing a null value, while the operator's admission webhook still required redisConfig to be an explicit empty object {} rather than null. Every installation attempt was rejected by the webhook. Since --set and -f flags could not inject this field through Helm's normal rendering path, the workaround was to use helm template to render the manifest to a file first, then patch the missing field by injecting redisConfig: {} directly into the YAML using sed before applying it with kubectl apply.

7.1.2 Fission Runtime and Ingestion Issues

Runtime Environment and Package Build Issues

One problem worth mentioning regarding Fission is the discrepancy between the local Python crawler and the runtime environment in Fission. While some code can run without any problems locally, it could fail when deployed as a Fission function because of the differences in Python version, package structure, or the runtime environment. Originally, the Bluesky pipeline deployed using the default Fission Python 3.7 environment. However, some crawler dependencies failed to work with this environment. Therefore, we created a Python 3.9 Fission environment. Furthermore, when the Bluesky pipeline was initially deployed, there were some problems when packages would take an unusually long time to build or timeout. After investigation, we discovered that the major cause of this problem was that the `requirements.txt` file contained heavy Python libraries like `atproto` and `pandas`. The solution involved removing the unnecessary libraries from the package, which helped us stabilize the package build process. Moreover, similar packaging problems should be addressed in the Reddit pipeline. The Reddit functions require the installation of libraries such as elasticsearch8, requests, flask, and vaderSentiment, which are declared in requirements.txt and requests through `build.sh`. In addition to this, it is important to ensure that the build script has proper permission to execute, or the package building process will fail.

Timeout and Workload Control

Timeout and workload control are other prominent issues in the ingestion pipeline. During the early testing of the Bluesky pipeline, if too many historical queries were processed in one execution, it often led to function timeouts. Subsequently, the crawler was modified to use small query batches, and historical crawling was split into multiple executions through cursor tracking.

The FuelWatch pipeline also encountered a similar problem. When processing a large number of historical CSV records, function execution often exceeded the default timeout. Therefore, the subsequent pipeline was changed to incremental ingestion, processing only a small number of CSV files in each execution and reducing the write pressure on Elasticsearch through bulk indexing.

For Reddit, the main timeout risk was the historical import workload. Instead of putting the full historical import into a Fission function, the pipeline separated realtime harvesting, bootstrap backfill, and deeper historical import. The deeper archive import was run as a local script through an Elasticsearch port-forward, which avoided forcing a long-running historical job into the Fission runtime.

Kubernetes Secret Overwrite

During the subsequent database configuration update process, the shared Kubernetes Secret "es-secret" got overwritten which caused the impossibility of receiving data from Bluesky. It happened due to the fact that when updating this secret in the mode `kubectl create ... --dry-run=client -o yaml | kubectl apply -f -`, the whole secret gets replaced rather than being updated only in one field. Since the newly generated secret did not include any credentials for logging into Bluesky, the trigger could still be executed; however, the crawler failed to log in and collect any information. At first, it was hard to determine where the problem occurred since the error might be related to secret configuration, API requests logic, function configuration, or even the connection to Elasticsearch. Finally, the error was identified while analyzing the secrets stored in Kubernetes, function logs, and Elasticsearch writes. Once the full secret "es-secret" including all the required fields was restored, the crawler started working properly again. This incident also indicates that the shared secret should be checked before updating it, as incomplete updates may affect multiple Fission functions.

API Request Limitations

During the repeated testing and timer-triggered execution processes of the Bluesky pipeline, another error "429 Too Many Requests" of API occurred, which caused the inability to write the data into ES. After analysis, it turned out that the problem was caused by too many logins and short intervals. Later, the issue was alleviated by changing the account, updating the Kubernetes Secret credentials, and extending the timer interval.

7.1.3 Elasticsearch Backend Query Issues

Elasticsearch Dynamic Mapping Conflict

During data source validation, we found mapping inconsistencies in fields such as `product_price` and `sentiment_score`. These inconsistencies caused some aggregation queries to return unexpected results. We identified the issue by running `GET /_mapping` queries in Kibana Dev Tools. The problem was then addressed during the reindex design phase.

Failed Join Approaches

Before choosing the final solution, we tested two join approaches at the database layer. The first approach used an ES|QL LOOKUP join. This failed because `social-posts.created_at` and the lookup key used different date formats. Elasticsearch also did not provide implicit date truncation for this case.

The second approach used an ENRICH policy. This was also abandoned for two reasons. First, the raw FuelWatch data used a forced default timestamp of 10:00:00, which made strict date-based joins fail. Second, the ENRICH processor returned only the first matching document instead of calculating a true daily FuelWatch average.

Because of these issues, we changed to a decoupled aggregation strategy. Each dataset is aggregated separately inside Elasticsearch. The results are then merged in Python using a date-keyed hash map.

UTC and AEST Timezone Misalignment

Social media posts were stored in UTC. This caused some posts to be grouped into the wrong local calendar date. The issue mainly affected posts created after 14:00 AEST, which were shifted into the next UTC day.

This created a consistent 10-hour mismatch between fuel price dates and social sentiment dates. We fixed it by explicitly setting `time_zone`: "Australia/Melbourne" in all `date_histogram` aggregations.

7.2 Data Collection Limitations

7.2.1 External Data Source Coverage

The limitations of data collection come mostly from the data sources that we collect data from externally. In the case of Reddit, there is a structural limitation on its public JSON API in that it only allows access to about 1,000 posts of each subreddit and does not allow queries for any particular date range or historical look back. As a result, even if we collected ten pages of posts per subreddit, we were only able to get posts from within the past 80 days. In the Bluesky pipeline, although both historical and real-time crawls can be done via the ``until`` and ``since`` cursors respectively, it is still required that the historical data be crawled gradually through repeated timer triggered executions.

The FuelWatch data is sourced from the API of Western Australia. The data is highly structured and has very good time continuity. Thus, it is ideal for use as the official fuel price baseline. Even though the data covers only the state of WA, the fuel prices in all states of Australia are determined based on similar factors such as global oil prices, tax rate, logistics costs, etc. Hence, we mainly use it as the official price trend benchmark, rather than attempting to precisely represent the oil prices in each state with it.

7.2.2 Keyword-Based Social Media Collection

The second limitation pertains to the keyword-based approach for data collection. In both Reddit and Bluesky, keywords including fuel prices, living costs, and those associated with Australia are used to collect relevant posts. This approach is quite simple to use, easy to understand, and helpful for future filtration and analysis. But there is no way that this technique can collect all implicit content as well. Users might not mention the pre-defined keywords in their discussions related to life stresses, transportation costs, or consumption pressures. These posts might not get included in the sample. On the other hand, some posts that might contain the keywords but have no semantic relation with the topics can also get included in the sample.

7.2.3 Location and Granularity Limitations

The location matching also has some restrictions. As Reddit and Bluesky do not offer latitude, longitude, and structured location information, this approach is capable of only estimating the ``matched_location`` column based on subreddit data, names of cities or states mentioned in the text, and location keywords related to Australia. This column can act as an additional feature that will enable rough grouping of areas, but it cannot be considered the real location of the user. It also restricts the possibility of comparing data with FuelWatch. The structure of the latter's data implies storing gas stations' names in capital letters, suburb name, and FuelWatch level. On the contrary, social media data offers only approximate city-level location features like ``Sydney`` or ``Melbourne``. Because of this inconsistency, exact geospatial matching proved impossible.

7.3 Analysis Limitations

Although the sentiment scores of Reddit and Bluesky by VADER are already obtained during the data ingestion process, the main issues with the two tools lie in the analysis part. VADER sentiment tool (Hutto & Gilbert, 2014) is famous for its ease of use and applicability to social media posts. It is a rule-based method relying on a dictionary. This means that the sentiment scores might not be accurate enough for sarcastic remarks, complex

sentiment detection in long messages, and some. Therefore, the sentiment results are more suitable as a reference for the overall trend analysis rather than a precise sentiment judgment for each post.

Topic indicators are based on keyword rules. These include fuel-related, cost-related, and Australia-related flags. They can be used to filter the results and organize them into certain groups. However, they are not a comprehensive topic model. There may be some other hidden or implicit topics that will not be detected. In addition, some posts that match the keywords may not be entirely relevant.

Any link between fuel prices and social media sentiment should also be read as correlation, not causation. Sentiment and discussion volume can change for many reasons. These may include wider economic conditions, political events, and media coverage. The visualisations are mainly used to explore trends. They should not be treated as proof of a direct causal link.

8. Fault Tolerance and Testing

8.1 Fault Tolerance and Remaining Failure Points

In our project, fault tolerance is not achieved through a complex failover system, but rather relies on several practical mechanisms to minimise the impact of failures. Firstly, Fission functions are themselves short-duration tasks. If a function execution fails, the timer trigger can still trigger the same function again after a certain period. This at least avoids the complete reliance on a single local machine for data collection, or on a crawler process that is always running.

Elasticsearch is another crucial recovery foundation. Once the data has been successfully written to Elasticsearch, subsequent backend aggregation, API queries, and Notebook analyses can continue to use this data. For FuelWatch, retaining `'fuelwatch-raw'` is also important because raw station-level records can serve as the basis for re-aggregation. If the calculation logic for the subsequent daily summaries needs to be adjusted, results can be generated anew based on the raw data instead of re-downloading all historical CSV files. The daily-aggregator function also benefits from this design. If an aggregation run fails on a given night, the raw station-level records in `fuelwatch-raw` remain intact. The aggregation can be re-triggered manually using `fission fn test --name daily-aggregator` without re-downloading any historical data.

The three-shard, one-replica configuration also contributes to fault tolerance. If one cluster node becomes unavailable, replica shards on the remaining node can continue serving read requests without data loss. Additionally, because all harvesters and API functions reference index aliases rather than physical index names, the underlying indices can be replaced or migrated without interrupting running services.

The recovery design of the Bluesky pipeline is more explicit. Since the Fission function does not save the local running state, the crawler progress cannot be simply stored in memory. Therefore, Bluesky uses `'bluesky-cursors'` to save the crawling progress. This way, even if the function is triggered again, the crawler can continue from the previous position instead of starting from the beginning every time. This design is not complex, but it is very crucial for a timer-based crawler.

Data duplication is also a factor that needs to be addressed. Both Reddit and Bluesky use document IDs unique to their platforms during the process of writing to Elasticsearch. In cases where the same post gets collected several times, Elasticsearch updates the existing document rather than creating duplicates. Similarly, the FuelWatch pipeline utilizes document IDs to ensure that there are no duplicated documents in case of safe repeated ingestion or rerun recovery. This approach adds to the stability of the pipeline during reruns, re-collecting data, or when the timer runs multiple times.

However, the existing solution uses a number of shared components, such as Elasticsearch, external data APIs, Fission Timer Triggers, and Kubernetes Secrets. These components were tested to ensure they function properly during the stages of deploying and testing the pipeline. Nevertheless, at this stage of the project, an automated failover for all of these components was not implemented due to its complexity. As such, the above-listed parts still represent possible improvements that need to be addressed in the future.

8.2 Testing and Validation

Our tests are mainly divided into several layers. The first layer is Fission function testing. The team manually triggers the functions using `'fission fn test'` to confirm whether the function can start, read the configuration, connect to Elasticsearch, and return reasonable results. For timer-triggered functions, this step is very important because the successful creation of the function does not necessarily mean that it can actually complete data collection.

The second layer is the validation of Elasticsearch and Kibana. We use the Elasticsearch count query to check if the data has been written to the target indices, such as `'social-posts'`, `'fuelwatch-raw'`, `'fuel-daily-summary'`, and `'bluesky-cursors'`. Kibana is mainly used to view the document structure, field names, timestamp fields, platform labels, and sentiment fields. This step is more reliable than only checking function output. It confirms that the data has actually reached the storage layer.

Field mappings were also checked with `GET /_mapping` queries. This confirmed that numeric fields such as `sentiment_score` and `product_price` were stored with the correct data types.

The third layer is API validation. Before deploying to the cloud, we tested the API logic locally with a FastAPI prototype, `api_gateway.py`. The auto-generated Swagger UI at <http://127.0.0.1:8000/docs> was used to inspect the JSON response schema. It also helped confirm that the merge logic worked before the API was moved to Fission. The final analytics API is `/api/v1/trends`. The team calls this endpoint through the port-forward tunnel of the Fission router. This is used to check whether the returned JSON includes fields such as date, fuel price, post count, average sentiment, and platform-level sentiment. This test confirms that the backend API can combine the fuel summary and social media aggregation into the format required by the Notebook. Function logs were also checked with `fission fn log` for both `fuel-sentiment-analytics` and `daily-aggregator`. This was the main way to diagnose runtime errors in the cluster, where direct console output is not available.

The last layer is Notebook validation. Jupyter Notebook will call the backend API and use Pandas and Plotly to generate the main charts. If the fuel price trend, discussion volume, sentiment trends, and platform comparison visualisations can be generated normally and the time range is consistent with the data in Elasticsearch, it indicates that the ingestion, storage, API, and visualisation layers are functioning consistently as an integrated pipeline.

In addition to these manual verifications, the GitLab CI/CD pipeline also provides some automated checks. It includes linting, unit tests, and dependency scanning. The tests cover two areas: basic project structure checks such as directory existence and key file presence, and core logic tests for the Reddit and Bluesky crawlers, including secret fallback, cursor handling, query batch rotation, function return structure, and error handling. However, these CI tests can only verify a part of the code logic and cannot completely replace the cloud deployment tests. Fission, Kubernetes Secret, Elasticsearch connection, and external API behaviour still need to be verified in the cluster environment. This is also a practical limitation in the testing of this project.

9. Conclusion

This project presents a cloud-native analytics platform for studying fuel price volatility in Australia and related public sentiment. Built on the NeCTAR Research Cloud, the system uses a multi-node Kubernetes cluster to run Fission ingestion functions, store data in Elasticsearch managed by ECK, and expose aggregated results through a RESTful API for analysis in Jupyter Notebook. This design separates data collection from analysis while keeping the workflow flexible.

The analysis combined daily FuelWatch data with ongoing Reddit and Bluesky discussions. The results show a moderate positive correlation (0.488) between fuel prices and discussion volume, suggesting that periods of higher prices tend to coincide with more online activity. In contrast, the correlation with overall sentiment is weak (-0.082), and this remains the case across a seven-day lag window. The two platforms also behaved differently in the collected data: Bluesky contributed more short, reactive posts, while Reddit discussions were longer and had slightly higher average sentiment.

From an engineering perspective, the system handled the collected data volume reliably during project testing. The modular Fission pipelines allow each harvester to run and be updated independently, while Elasticsearch sharding, index aliases, and pre-aggregated summaries helped keep API queries efficient over more than four million records.

Several limitations remain. VADER is lightweight and easy to deploy, but it does not handle sarcasm, Australian slang, or context-dependent sentiment particularly well. Location information is also coarse, which limits deeper spatial analysis. Future work could improve sentiment modelling with a more context-aware approach, use Named Entity Recognition to extract more detailed locations, add stronger alerting and recovery mechanisms, and extend the ingestion layer with supermarket pricing data or official inflation indicators.

LLM Usage Declaration

Large Language Models were used during the development of this project primarily for some code debugging assistance. All system design, implementation, deployment, analysis, and final verification were completed and reviewed by the project team. A detailed LLM usage declaration is included separately in the repository under the ``llm/`` directory.

References

- Balduf, L., Sokoto, S., Ascigil, O., Tyson, G., Scheuermann, B., Korczyński, M., Castro, I., & Król, M. (2024). Looking AT the blue skies of Bluesky. *Proceedings of the 2024 ACM Internet Measurement Conference*, 76–91. <https://doi.org/10.1145/3646547.3688407>
- Carrión, C. (2022). Kubernetes as a standard container orchestrator: A bibliometric analysis. *Journal of Grid Computing*, 20(4), 42. <https://doi.org/10.1007/s10723-022-09629-8>
- Failla, A., & Rossetti, G. (2024). "I'm in the bluesky tonight": Insights from a year worth of social data. *PLOS ONE*, 19(11), e0310330. <https://doi.org/10.1371/journal.pone.0310330>
- Government of Western Australia, Department of Mines, Industry Regulation and Safety. (2026). Historic fuel prices. *FuelWatch*. <https://www.fuelwatch.wa.gov.au/retail/historic>
- Hong, D., Fu, Z., Zhang, X., & Pan, Y. (2025). Research on the development and application of the GDELT event database. *Data*, 10(10), 158. <https://doi.org/10.3390/data10100158>
- Hutto, C. J., & Gilbert, E. E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media*, 216–225. <https://ojs.aaai.org/index.php/ICWSM/article/view/14550>
- Jain, R., Kumar, A., Nayyar, A., Dewan, K., Garg, R., Raman, S., & Ganguly, S. (2023). Explaining sentiment analysis results on social media texts through visualization. *Multimedia Tools and Applications*, 82(15), 22613–22629. <https://doi.org/10.1007/s11042-023-14432-y>
- Mondal, S. K., Zheng, Z., & Cheng, Y. (2024). On the optimization of Kubernetes toward the enhancement of cloud computing. *Mathematics*, 12(16), 2476. <https://doi.org/10.3390/math12162476>
- Palade, A., Kazmi, A., & Clarke, S. (2019). An evaluation of open source serverless computing frameworks support at the edge. *Proceedings of the IEEE World Congress on Services (SERVICES)*, 226–231. <https://doi.org/10.1109/SERVICES.2019.00057>
- Ríos-Monje, C., Parra-Royón, M., Moldón, J., Sánchez-Expósito, S., Garrido, J., Darriba, L., & Salgado, J. (2023). An approach to provide serverless scientific pipelines within the context of SKA. *arXiv*. <https://arxiv.org/abs/2306.09728>